

<for Replit: Создать структуру проекта с разделением на backend/, frontend/, workers/>

<for Replit: Все переменные окружения вынести в Secrets и Configurations>

<for Replit: Использовать development и production режимы с разными БД>

<for Replit: Настроить логирование в development_logs/development_journal.md>

<for Replit: Gemini API проксировать через AI_PROXY_GEMINI (SOCKS5)>

«Название продукта»

Пояснение к названию продукта: Раздел содержит официальное наименование проекта, которое должно быть кратким, информативным и отражать суть инициативы. Это позволяет унифицировать идентификацию документа в корпоративной системе документооборота

Оглавление

1.	Концепция продукта/краткое описание	1
2.	Ключевые аспекты сервиса/ Предыстория	2
3.	Монетизация:	4
4.	Функциональные требования	6
4.1.	Клиентский путь/ user journey	6
4.2.	Дизайн-проект продукта	6
4.3.	Компоненты UI	7
4.4.	(Пример главы с одной из функций продукта) Быстрый и простой анализ	7
4.5.	Админ панель	9
5.	Нефункциональные требования	12
6.	Архитектура	13
7.	Технологический стек	15
8.	Рекомендации для написания кода (к производству приложения), например, Replit.	18
9.	Техническое задание	23

1. Концепция продукта/краткое описание

Описание пункта: Краткое определение, что это за сервис/продукт, какую проблему он решает и какие основные функции предоставляет.

Заполнить:

- Тип продукта (веб-приложение, мобильное приложение, API, бот и т.д.)
- Краткое описание сути (1-2 абзаца максимум)
- Список основных функций (3-5 пунктов)

Пример:

Сервис — AI-ассистент для анализа идей стартапов, доступный в Telegram. Пользователь описывает идею, получает комплексный анализ реализуемости, финансовую модель и рекомендации по продвижению за минуты вместо месяцев экспертизы.

StartupRoasting — AI-powered платформа для анализа и оценки бизнес-идей стартапов. Система помогает предпринимателям структурировать идеи, получать обратную связь по 5 ключевым категориям и автоматически генерировать профессиональные презентации для инвесторов. Доступна через веб-интерфейс и Telegram-бота.

2. Ключевые аспекты сервиса/ Предыстория

Описание пункта:

Раздел объясняет исторический и бизнес-контекст проекта, включая мотивацию, целевые проблемы и стратегические цели. Это помогает обосновать необходимость проекта и установить рамки для последующих требований.

Целевая аудитория:

Описание пункта: Кто будет использовать продукт? Какие сегменты пользователей?

Заполнить:

- Первичная аудитория
- Вторичная аудитория (если есть)

Пример:

Начинающие предприниматели, фаундеры стартапов, студенты; также инвесторы для первичной оценки.

Решаемая боль:

Описание пункта: Какую проблему решает продукт? В чём боль пользователя?

Заполнить:

- 3-4 основных проблемы, которые решает продукт

Пример:

- *Недостаток быстрой экспертизы для валидации идей*
- *Длительный анализ (месяцы вместо минут)*
- *Дорогие консультации экспертов*
- *Высокий риск провала без данных*

уТП/уцп:

Описание пункта: Что делает ваш продукт уникальным и привлекательным?

Заполнить:

- 2-3 ключевых преимущества перед конкурентами

Пример:

- *Мгновенный анализ через привычный мессенджер (Telegram/Discord)*
- *Готовый комплект документов для инвесторов*
- *Персонализированная база знаний, не требующая подписки на каждый инструмент*

Область применения с точки зрения бизнеса:

Описание пункта: Как и где используется продукт в бизнесе?

Заполнить:

- Типы заказчиков / компаний
- Процессы, в которые встраивается
- Выгода для бизнеса

Пример:

Заказчики: Стартап-студии, инкубаторы, банки, телеком компании

Процесс: Генерация и валидация идей, оценка R&D проектов

Выгода: Экономия месяцев аналитики, снижение рисков инвестиций, масштабирование без затрат

Оценка возможностей по обеспечению новых потребностей бизнеса:

Описание пункта: Как обстоит дело сейчас и как будет после вашего продукта?

Пример:

AS IS (как сейчас):

- *Длительный и дорогостоящий анализ*
- *Высокий риск провала*
- *Недоступность экспертизы*
- *Фрагментированность данных*

TO BE (как будет):

- *Дает инсайты за минуты, а не месяцы*
- *Автоматизирует рутину*
- *Масштабируется без дополнительных затрат*
- *Открывает новые возможности:*
 - *Компании могут запустить внутренний инкубатор идей,*
 - *Инвесторы и акселераторы могут использовать сервис для первичной фильтрации заявок*
 - *Предприниматели получают возможность экспериментировать с десятками идей*
 - *Появляется возможность выхода на новые рынки через быструю локализацию анализа под разные ниши и регионы.*

3. Монетизация:

Описание пункта: Как продукт приносит доход?

Заполнить:

- *Модель монетизации*
- *Цены / тарифы (если определены)*

Пример:

- *Freemium: базовый анализ бесплатно, премиум-подписка за €5/месяц*
- *Pay-per-use: дополнительные анализы за €2*

- *B2B: интеграция в компании от €500/месяц*

4. Функциональные требования

Верхнеуровневое описание основных функций. Перечислите ключевые сценарии использования системы в формате списка.

Этот раздел детализирует функциональные требования через описание типичных сценариев взаимодействия пользователей с системой. Он фокусируется на пользовательских потоках, ролях и ожидаемых результатах, обеспечивая четкое понимание функциональности для разработчиков и тестировщиков.

Пример:

- Пользователь регистрируется через email или Telegram
- Пользователь выбирает режим анализа (LITE/FULL/INVESTOR)
- Пользователь отвечает на вопросы AI-ассистента (до 12 основных + уточняющие)
- AI формулирует ответы через подвопросы, помогая пользователю
- Система оценивает идею по 5 категориям (Проблема/Решение, Рынок, Бизнес-модель, Команда, Планы)
- Пользователь получает итоговый отчёт с рекомендациями
- Пользователь генерирует Pitch Deck презентацию (PRO)
- Пользователь сохраняет проекты в личном кабинете
- Администратор управляет пользователями, подписками и настройками системы
- Telegram-пользователь может привязать аккаунт к веб-версии

4.1. Клиентский путь/ user journey

Описание пункта: Опишите последовательность шагов пользователя или предоставьте ссылку на дизайн-проект.

Подраздел описывает клиентский/пользовательский путь (user journey), включая визуальные и навигационные аспекты. Ссылка на внешние ресурсы (например, Figma) позволяет интегрировать детальные макеты без перегрузки документа.

4.2. Дизайн-проект продукта

Описание пункта: Подраздел предоставляет доступ к дизайн-материалам проекта, которые необходимы для front-разработки. Ссылка на внешние ресурсы (например, Figma) позволяет интегрировать детальные макеты без перегрузки документа.

Также можно вставить уже готовый корпоративный бренд-бук и шаблон дизайна компании с корпоративными цветами и описанием оформлени.

4.3. Компоненты UI

Описание пункта: Опишите ключевые UI-элементы и их поведение в табличном формате. Подраздел детализирует интерфейсные компоненты, их функциональность и поведение, способствуя единообразию дизайна и облегчая реализацию front-разработки

Пример:

Элемент	Поведение
Кнопка «Начать анализ»	Переход в чат, выбор режима через URL параметр <code>?mode=lite/full</code>
Чат-интерфейс	Автоскролл, <i>skeleton-loader</i> при загрузке AI-ответа
Прогресс-бар категорий	Реактивное обновление при получении оценок
Модальные окна	<i>Paywall</i> для FULL режима, подтверждение действий
Мобильная версия	Адаптивный дизайн, полноэкранный чат
Состояния кнопок	<i>disabled</i> при загрузке, <i>loading spinner</i> при отправке
Telegram Bot	Inline-кнопки для навигации, <i>callback-обработчики</i>

4.4. (Пример главы с одной из функций продукта) Быстрый и простой анализ

Описание пункта: что делает эта функция? Какую задачу решает?

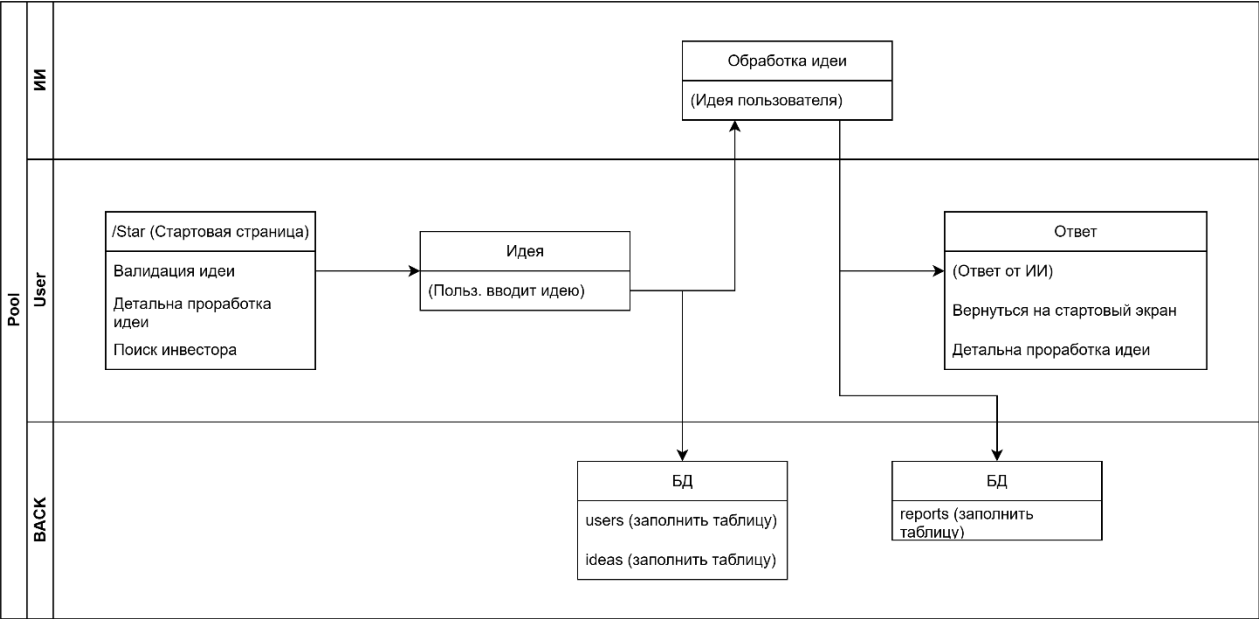
Заполнить:

- Более сложная функция с диалогом, уточняющими вопросами и т.д
- Ключевые требования (3-5 пунктов)
- Примеры использования

Требование	Описание
------------	----------

Вход пользователя	Пользователь инициирует [действие]. Пример: нажимает кнопку "Анализ", вводит текст
Обработка	Система [обрабатывает данные]. Пример: валидирует ввод, отправляет на AI
Вывод	Система возвращает [результат]. Пример: структурированный отчёт в виде сообщения
Сохранение	Результат сохраняется [куда]. Пример: в таблицу reports БД
Завершение	После завершения пользователю предлагается [действие]. Пример: скачать отчёт или создать презентацию

Пример схемы взаимодействия:



Функц. Валидация
идеи.drawio

4.5. Система аутентификации и авторизации (ролевая модель)

Описание пункта: Описание системы аутентификации и авторизации (ролевой модели).

Пример:

Необходимо реализовать ролевую модель руководствуясь требованиями:

Система аутентификации и авторизации	<i>OAuth2 — протокол авторизации для делегирования доступа (токены вместо паролей).</i> <i>Реализовать двухтокенную систему: JWT</i> • <i>Access Token</i> • <i>Refresh Token</i> <i>Обязательная верификация Email</i> <i>Возможность сбросить пароль</i>
ролевая модель	<i>Простая ролевая модель из 2х ролей:</i> <i>user (пользователь)— обычный пользователь продукта.</i> <i>admin (Администратор)— тех/бизнес-владелец: все полномочия.</i> <i>admin (Администратор) может:</i> - <i>Полный доступ без проверки подписки</i> - <i>Видит кнопку «Админ панель» и имеет доступ к панели администратора</i> - <i>Может назначать других администраторов</i> - <i>Может выдавать бесплатную подписку</i>
Поддерживаемые методы входа	- <i>OAuth2 через Google</i> - <i>Yandex</i> - <i>Email + Password</i>
Личный кабинет	<i>Личный кабинет каждого пользователя с полезной информацией</i>
Монетизация	<i>У каждого пользователя должна быть возможность оформить подписку. Более подробно описано в блоке «Монетизация»</i> <i>Простая модель подписок:</i>

	<i>1 неделя, 1 месяц, 3 месяца, 1 год. Без ограничений на функционал.</i> <i>Админская подписка (Безлимит и бесплатно на все)</i>
--	--

4.6. Админ панель

Описание пункта: описание панели администратора, как правило в веб-интерфейсе, которая позволяет управлять системным аналитиком (Админом) управлять системой без привлечения разраба для правки кода.

Панель предназначена для мониторинга активности, управления данными и настройки системных параметров без необходимости изменения кода или перезапуска сервиса.

Как правило в админ панель добавляют функции, которые потенциально, с определённой периодичностью, могут изменяться непосредственно в самом коде и базе данных.

Разработчик должен максимально вынести в эту панель переменные продукта, чтобы не хардкодить важные параметры.

ИИ анализирует хостинги (AIOps): мониторит метрики (CPU/трафик), предсказывает сбои, строит отчёты. Примеры: Improvado AI Agent генерирует PDF с KPI/графиками по расходам; CloudMonitor.ai прогнозирует downtime по логам, оптимизирует ресурсы (экономия 20-30%).

Пример:

<i>Требование</i>	<i>Описание</i>
<i>Пользователи</i>	<i>Вкладка «Пользователи»:</i> <i>Отображаются пользователи с логином (логин/почта), датой создания УЗ (дата регистрации), подписка, дата окончания подписки, роль, Заблокирован ли пользователь и другая полезная информация о пользователе.</i> <i>Функции администратора:</i> - Назначить админом - Убрать админа - Выдать подписку - Заблокировать/разблокировать <i>- Поиск</i>

	- Фильтрация - Пагинация - Удалить пользователя
API банка для оплаты/подписка	Вкладка «Платежи»: Возможность правки файла конфигурации банковских реквизитов для оплаты подписок (монетизации). Возможность изменять сумму стоимости подписки/тарифа.
Управление промтами	Возможность просмотра, редактирования и сохранения системных промтов для каждого режима (быстрый анализ, расширенный функционал, поиск инвестора, генерация презентации). Изменения применяются немедленно
Управление API LLM модели	Настройка: имя модели (например, Grok, Gemini), адрес API, параметры генерации (temperature, max_tokens). Возможность тестового вызова для проверки работоспособности
Управление токенами Telegram	Добавление, редактирование и удаление токенов ботов. Поддержка нескольких ботов (например, тестовый и боевой). В том числе определение чат-бота для отправки ошибок при журналировании.
Экспорт данных таблиц	Выгрузка данных из базы в формате CSV Chart.js — JavaScript-библиотека для создания интерактивных HTML5-графиков (линейные, бары, пироги). Plotly — кросс-платформенная библиотека (Python/JS) для научных графиков, 3D, карт, с экспортом в интерактивный HTML. Pandas — Python-библиотека для DataFrame (таблицы данных), анализа CSV/Excel
Журнал событий	Отображение логов в реальном времени
SMTP	Единый почтовый ящик для отправки подтверждающих кодов
Мониторинг активности AI сервисов	Реализуй проверку работоспособности AI сервисов (в админ-панели, если не указано другое в ТЗ). Сервисы должны проверять фактическое подключение к api. Должен отображаться статус (доступен / недоступен) и, если статус недоступен - отображать код ошибки и сообщение ошибки.

4.7. Интеграция с сервисом оплаты

Описание пункта: тут описываются как продукт интегрируется с системами оплаты.

Пример для РобоКассы:

Реализуй интеграцию с Робокассой для оплаты подписки | 'Базовая' | 'Стандартная' | 'Максимальная' | 'Ультра' | 'Энтерпрайз' для пользователя. (цены на подписки указываются в админ - панели)

Данные для подключения магазина создай и используй:

Общие переменные (добавь в secrets проекта SHOP_ID_RK и в configurations HASH, RESULT_URL_RK, SUCCESS_URL_RK, FAIL_URL_RK)

SHOP_ID_RK=

HASH=MD5

RESULT_URL_RK=

SUCCESS_URL_RK=

FAIL_URL_RK=

Для production режима создай и используй ключи магазина(добавь эти ключи в secrets проекта)

PASS1_RK=

PASS2_RK=

Для development режима используй (ключи для тестового магазина). Эти ключи должны храниться в переменных окружения, также добавь их в secrets проекта

PASS1_TEST_RK=

PASS2_TEST_RK=

Управление настройками Робокассy в Админ Панеле:

- изменять тип платежа тестовый на боевой (тестовый использует ключи PASS1_TEST_RK и PASS2_TEST_RK из переменных окружения, для боевого использует PASS1_RK и PASS2_RK также из secrets проекта.)

5. Нефункциональные требования

Пример описания:

Требование	Описание
Производительность	— Время ответа на запрос: не более 3 секунд от отправки идеи до получения анализа. — Обработка параллельных запросов: поддержка до 100 одновременных сессий. — Время генерации презентации: не более 60 секунд от запроса до получения ссылки
Доступность	— Система должна быть доступна 24/7. — При сбое — автоматический перезапуск сервиса
Масштабируемость	
Безопасность	CSRF защита Все ключи API и секреты только в переменных окружения (.env), никогда не в коде Rate limiting по IP для защиты от ботов
Надёжность	— Резервное копирование базы данных: ежедневно, хранение — 7 дней. — Логирование всех ключевых событий (запросы, ошибки, оплаты) в централизованную систему — Оповещения о сбоях из логирования (Telegram-бот для админов).
Поддерживаемость	— Логика промтов вынесена в конфигурационные файлы — обновление без перезапуска.
Совместимость	— Поддержка всех актуальных версий Telegram (включая Telegram Desktop, iOS, Android). — Веб-интерфейс: поддержка Chrome, Firefox, Safari. — Адаптивный дизайн для мобильных устройств.
Юзабилити	— Интерфейс интуитивно понятен без инструкций. — Все действия — через кнопки, минимум ручного ввода.
Интегрируемость	
Поддерживаемость админ-панели	Интерфейс админ-панели должен быть интуитивным

6. Архитектура

Описание пункта: Как устроена система? Какие компоненты взаимодействуют?

Заполнить:

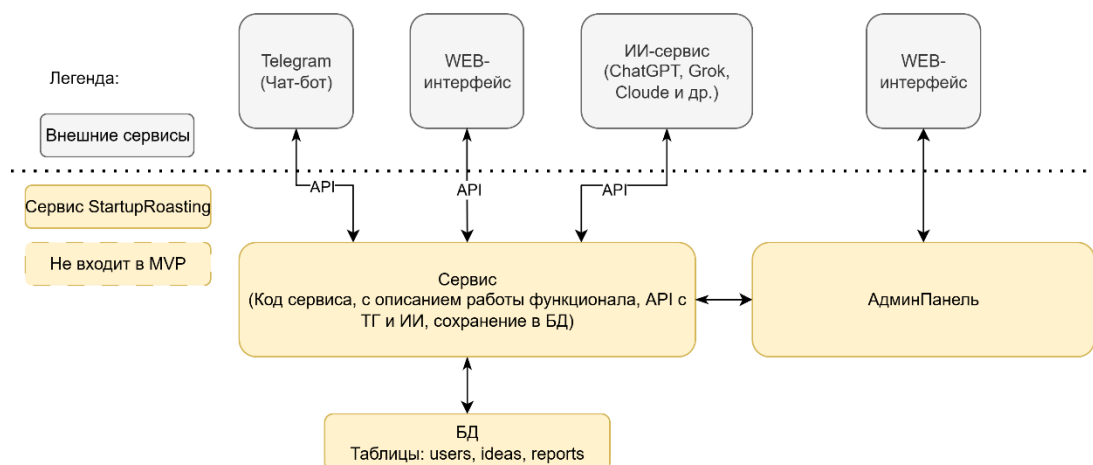
- Текстовое описание архитектуры (1-2 абзаца)
- Диаграмма архитектуры (если есть)
- Ключевые компоненты:
 - Фронтенд (веб, мобиль, ботчик и т.д.)
 - Бэкенд (API, обработка, логика)
 - БД (где хранятся данные)
 - Внешние сервисы (платежи, СМС, AI и т.д.)

Пример:

Компоненты:

- Telegram Bot — точка входа для пользователей
- React Web App — веб-интерфейс (зеркальный функционал)
- Python Backend — обработка всей логики, работа с AI
- PostgreSQL — хранилище данных пользователей и анализов
- Gemini AI API — генерация анализов
- Gamma API — создание презентаций

Пример схемы:



Архи

StartupRoasting.dra

7. Технологический стек

Описание пункта: Какие технологии и инструменты используются?

Подраздел перечисляет используемые технологии по уровням системы, обеспечивая прозрачность выбора стека для оценки совместимости, масштабируемости и поддержки.

Заполнить:

- Бэкенд (язык, фреймворк)
- Фронтенд (язык, библиотеки)
- База данных
- Хостинг / облако
- Дополнительные сервисы (AI, Презентации, Мессенджер)

Пример заполнения:

- *Бэкенд: Python + FastAPI*
- *Фронтенд: React*
- *База данных: PostgreSQL*

Frontend Web:

- *JavaScript — язык программирования. На нём основан весь фронтенд.*
- *TypeScript — надстройка над JavaScript с типами (расширение языка).*
- *React — библиотека (инструмент) для интерфейсов, написанная на JavaScript/TypeScript.*
- *Next.js — фреймворк (надстройка) поверх React для веб-приложений.*

Frontend Mobile:

- *Адаптивный web*
- *PWA - (установка на телефон как «приложение», оффлайн-режим на базовом уровне*

Backend:

- *TypeScript*
- *Node.js - среда, которая позволяет запускать JavaScript/TypeScript*

- *NestJS - это фреймворк поверх Node.js + TypeScript, который даёт структуру приложения: модули, контроллеры, сервисы, DI*
- *Python*

API Gateway:

- *Ничего. NestJS - справится без сложных Kong / AWS API Gateway*

База данных (структурированные данные и запросы):

- *PostgreSQL — основная реляционная БД.*
- *Elasticsearch — для поиска.*
- *ClickHouse — для аналитики*
- *Для кэша не использовать пока Redis, с этим справится PostgreSQL.*

Хранилище (файлы и «тяжёлый» контент):

- *S3-совместимое объектное хранилище*

Платежи:

- *ЮKassa, РобоКасса, ВТБ через СБП*

Авторизация:

- *OAuth 2.0*
- *Replit Auth не подойдет для нашего масштабного проекта.*

Логирование:

- *NestJS Logger + pino?*

Мониторинг:

- *Логи/ошибки: Sentry (?)*
- *Метрики: Prometheus + Kibana + Grafana (в простой конфигурации).*
- *Prometheus - Система сбора метрик (числовых показателей). Это временная БД (Time Series Database) — БД специально для метрик.*

- *Kibana - информация о логах, инструменты для визуализации данных, тесно связанная с Elasticsearch (база данных для логов), в основном используется для анализа журналов и данных временных рядов.*
- *Мониторинг Grafana - инструменты для визуализации данных, больше подходит для мониторинга метрик из различных источников данных.*

DevOps:

- *Репозиторий и CI/CD: GitHub*
- *Хостинг, VPS: TimeWEB*

8. Рекомендации для написания кода (к производству приложения), например, Replit.

В данном техническом задании есть рекомендации для Replit как в примере: <for Replit: какая_то_рекомендация>

Воспринимай эти рекомендации как руководство к выполнению и добавь это упоминание в самом начале для документа в формате MD.

> Любые рекомендации для Replit можно указывать непосредственно в ТЗ, главное сделать одинаковый шаблон для каждого указания и ставить команду, например: <for Replit: какая_то_рекомендация>

1. Журналирование процесса разработки и развития проекта

Создай систему детального логирования всех операций. При расширении функционала приложения и изменении кода всегда сопровождай изменения системой логирования новых процессов

При старте проекта создай файл `development_journal.md` в директории `development_logs/`

После реализации каждого функционального требования, расширении функционала приложения и изменении кода добавляй в файл `development_journal.md`:

- Этап разработки (указывается в начальном ТЗ)
- Описание функционала
- список задач и подзадач, выполненных с последнего обновления `development_journal.md`
- чек-боксы проверки реализованного функционального требования
- краткое описание по тестированию каждого функционального требования
- отметки о прогрессе, датах, исполнителях.

> Правило: этот файл должен быть обновлён и заполнен до перехода к следующему этапу разработки, иначе этап считается не завершённым.

Так же создай журнал развития проекта. Назови его `replit_journal.md` в директории `development_logs/`. Самостоятельно вноси в него любые новые опции и функции. Также вноси в журнал любые мысли пользователя на перспективу развития проекта.

2. Запрет на хард-кодинг значений

Вынеси в `secrets` и `configurations` проекта все переменные и константы:

- Секретные ключи, токены, логины, пароли, любые переменные окружения
- Все переменные в проекте, используемые для подключения / взаимодействия с внешними интеграциями (AI сервисы, SMTP сервер, проху, внешние поисковые системы, внешние базы данных, системы сбора логов, системы мониторинга, Телеграм бот и тому подобное)

> Правило: все переменные окружения должны быть выведены в secrets и в configurations проекта.

Создай файл .env.secrconfig со всеми переменными окружения, используемыми в проекте. Определи эти переменные из возможности переноса проекта на внешний VPS.

3. Логирование как обязательный элемент системы

Настрой в проекте комбинированное логирование для Sentry и Loki + Grafana. Добавь переменные окружения в secrets и configurations проекта и в .env.secrconfig

Логирование действий администратора. Логировать:

- Назначение admin
- Снятие admin
- Выдачу подписки
- Изменение цены
- Изменение платежных ключей

4. Подключение внешней БД (для Postgres)

База данных должна быть доступна по переменной окружения DATABASE_URL. Вынеси DATABASE_URL в secrets проекта.

5. Подключение к production DB в Реплит

Для режима разработки используй development database. Для опубликованного приложения используй production database.

6. Подключение к Telegram bot (Если есть боты)

Используется 2 телеграм бота TEST_TLG_BOT и PROD_TLG_BOT. При работе в development режиме используй TEST_TLG_BOT (вынеси в secrets проекта эту переменную). При работе в production режиме используй PROD_TLG_BOT (вынеси в secrets проекта эту переменную).

7. Создание документации на русском языке (Для сопровождения проекта, чтобы быстро разобраться как все работает при необходимости)

- Создай Архитектурное описание (SAD) (Описание структуры системы, модулей, используемых технологий и взаимодействия с базами данных.)
- Описание API (Документация для интеграции. Например, Swagger/OpenAPI)
- Пользовательскую документацию (Руководство пользователя, Заметки к релизу)

- Руководство администратора (Как настраивать права доступа, делать бэкапы, мониторить состояние сервера и устранять критические ошибки.)
- Инструкция по безопасности (Описание протоколов шифрования, хранения паролей и мер защиты данных.)
- Файл README.md (Краткий «входной билет» в проект: как запустить, как собрать, основные зависимости.)
- Инструкцию для работы с миграцией базы данных.

8. Расширенная диагностика в процессе разработки

Реализуй расширенную диагностику с выводом в логи приложения для всех используемых внешних интеграций и работы с БД. Расширенная диагностика должна работать в development режиме Replit и должна отключаться при работе приложения в production.

9. Подключение AI сервисов, если используются в проекте

Соединение со AI сервисами:

Все AI сервисы которые НЕ указаны ниже используют прямое подключение.

1. Соединение с:

Openai api

Anthropic

Perplexity

Microsoft Azure OpenAI

Amazon Bedrock

проводит через http proxy AI_PROXY=http://kamerton:kamerton@31.130.131.48:3128

2. Соединение с:

Gemini

проводит через socks5h xray reality AI_PROXY_GEMINI=socks5h://172.17.0.1:10808 (xray клиент установлен непосредственно на хосте vps) используй подсказку ниже

"Вот как было реализовано проксирование запросов к Gemini через SOCKS5 прокси:

Проблема: Google Gemini API блокирует запросы из России. Стандартный SDK @google/genai не поддерживает прокси.

Решение: Заменяли SDK на прямые HTTP-запросы через axios, который поддерживает прокси.

Шаги:

Убрали SDK — вместо @google/genai используем axios для прямых REST-запросов к API Gemini по адресу:

`https://generativelanguage.googleapis.com/v1beta/models/{model}:generateContent`

Добавили SOCKS5 прокси-агент — через пакет socks-proxy-agent. Если задана переменная окружения AI_PROXY_GEMINI, создаётся SOCKS5 агент:

`AI_PROXY_GEMINI=socks5h://172.17.0.1:10808`

Адрес 172.17.0.1 — это IP Docker bridge, через который контейнер обращается к храу, запущенному на хосте VPS.

Axios с прокси-агентом — при каждом запросе к Gemini в параметры axios передаётся httpAgent и httpsAgent с этим SOCKS5 агентом. Если прокси не задан — запросы идут напрямую.

Для OpenAI/Perplexity — используется отдельный HTTP-прокси через переменную AI_PROXY_URL (обычный HTTP прокси на другом сервере).

Конфигурация на VPS:

храу слушает на 172.17.0.1:10808 (не на 127.0.0.1, чтобы Docker-контейнер мог достигать)

В .env задаётся `AI_PROXY_GEMINI=socks5h://172.17.0.1:10808`

Итого: Gemini-запросы из контейнера идут через храу SOCKS5 прокси на хосте, который перенаправляет трафик в обход блокировки. "

Вынеси переменные AI_PROXY_GEMINI и AI_PROXY в secrets проекта и проверь их наличие в .env.example и в docker-compose.yml

Создай переменные окружения для токенов AI сервисов и вынеси их в secrets проекта. Проверь их наличие в .env.example и в docker-compose.yml

9. Техническое задание

Описание пункта: Раздел содержит детальный иерархичный план задач и чеклисты, включая описание, вехи и зависимости. Ссылка на внешние инструменты (например, таблицы в облаке) позволяет поддерживать актуальность без редактирования основного документа.

Пример заполнения:

Этапы разработки:

Этап	Выполнено (Разработчик)	Протестировано разработчиком	Комментарий
1. Подготовить Бэкенд. Сделать проект в GitHub.	Готово		Создать приватный репозиторий. Настроить структуру проекта в предоставленном Git-e
2. Сделать бота	Тест		
• Зарегистрировать бота в Telegram (BotFather)			Получить токен.
3. Настроить сервис с LLM			
• Сделать аккаунт (Gemini)			Указать УЗ, API
• Получить и протестировать API			Проверить вызов LLM (например, через Postman).
• Загрузить промт для разных режимов (быстрый, расширенный, инвестор)			Промт хранится в конфиге/переменных окружения.
4. Построить БД (PostgreSQL)			
• Создать и настроить базу данных			
• Создать таблицы: `users` (id, telegram_id, subscription), `ideas` (id, user_id, idea_type, idea_text), `reports` (id, idea_id, analysis)			Убедиться, что связи корректны.
5. Написать бэкенд			
• Быстрый анализ			
• Расширенный			

• Инвестор			
• Админ панель			
• Сохранение идеи и анализа в БД			
• Генерация презентаций через Gama			
6. Синхронизация			
• Подключить Telegram Bot API к бэкенду			Бот должен принимать сообщения и отправлять ответы.
• Синхронизовать БД с бэком			
• Настроить вызов LLM и получение ответа			Ответ парсится и передаётся пользователю.
7. Протестировать работу сервиса/интеграционное тестирование			
• Проверить все три режима в Telegram			Вручную или через простые сценарии.
• Убедиться, что данные сохраняются в БД			Проверить записи в таблицах
• Проверить корректность анализа от ИИ			Логика, структура, «Первый шаг».
8. Написать фронт (React)			
• Создать React-приложение			
• Сделать веб-интерфейс с теми же функциями, что и в Telegram, в виде диалогового окна			Полная копия Telegram-бота.
9. Разработать админ-панели			
• Управление промтами			
• Управление API LLM модели			
• Управление токенами Telegram			
• Экспорт данных таблиц			

10. Журналирование и уведомления • Настроить логирование событий • Создать отдельного Telegram-бота для уведомлений • Реализовать отправку ошибок в Telegram • Поддержка нескольких Telegram-ботов			
11. Развернуть на хостинге			Доступ по ссылке.

Критерии приемки:

Заполнить:

- Общие критерии (список чек-пунктов, которые должны быть выполнены)
- Критерии для каждой основной функции

Пример:

1. Система принимает ввод текста и отправляет анализ согласно функционалу
2. API корректно интегрируется с AI-сервисом
3. Данные сохраняются в БД без потерь
4. Веб-интерфейс работает на всех основных браузерах
5. Telegram бот отвечает на команды без задержек
6. Админ-панель позволяет редактировать и управлять системой
7. Документация полная и понятная
8. Нет критических ошибок при нагрузочном тестировании

Примечания:

- Все сроки указываются в трудо-часах или календарных днях
- При необходимости добавить новые разделы (например, Security, Compliance, Analytics)
- Обновлять этот документ по мере выявления новых требований
- Синхронизировать с командой разработки на регулярных встречах